

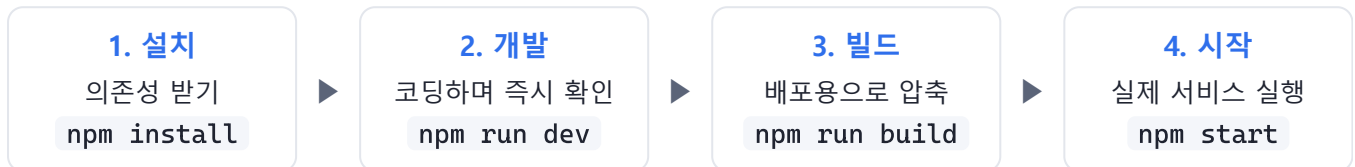
Next.js 개발 입문 가이드

초보자용

npm install부터 화면 디자인까지 — 처음 시작하는 사람의 시점에서 단계별로 정리

요약: 전체 흐름 한눈에 보기

웹사이트 하나를 Next.js로 만들 때, 코드를 쓰기 시작해서 화면이 뜨고 진짜 서비스로 배포되기까지는 보통 아래 네 단계를 거칩니다.



비유 — 요리에 비유하면 ① 장보기(install) → ② 시식하며 만들기(dev) → ③ 포장(build) → ④ 손님상에 내기(start)입니다. 평소 코딩하는 동안에는 ①②만 반복하고, ③④는 실제 서비스를 띄울 때만 합니다.

결론부터: 네, 이 흐름이 개발의 “기본”이 맞습니다. 다만 “기본”은 *도구를 다루는 기본 동작*이고, **실제 개발은 그 안에서 코드를 짜는 일**입니다. CSS/TSX는 그 안의 “디자인·동작” 영역이고요. 아래에서 단계별로 풀어 설명할게요.

1단계 · 프로젝트 시작하기 (설치)

1-1. npm install 은 뭘 하는가

프로젝트가 “어떤 부품들을 쓰는지”는 `package.json` 에 적혀 있습니다. `npm install` 은 그 목록을 읽어서 **인터넷에서 부품들을 받아 node_modules 폴더에 모아두는 작업**입니다.

```
# 한 번만 해도 되는 일
npm install      # = package.json의 모든 의존성을 설치
npm install zod  # = 새 부품(zod) 추가 설치
```

비유 — `package.json` 은 **쇼핑 목록**, `node_modules` 는 **창고**. install 명령은 “목록 보고 창고 채우기”입니다.

1-2. 한 번만 하면 되나요?

- 프로젝트를 처음 받아왔을 때 → **한 번**
- 새 부품(라이브러리)을 추가했을 때 → **그때 또**
- `package.json` 이 바뀐 사람의 코드를 git pull 받았을 때 → **그때 또**

주의 `npm` · `pnpm` · `yarn` 은 같은 일을 하는 다른 도구입니다. 한 프로젝트에서는 **하나만** 골라서 써야 해요. 섞으면 잠금파일(lockfile)이 충돌해 설치가 깨집니다.

2단계 · 개발하며 즉시 확인하기

2-1. npm run dev 의 의미

npm run dev 는 개발 전용 서버를 띄웁니다. 코드를 저장하면 브라우저가 알아서 새로고침되어 결과가 즉시 보입니다(이걸 HMR, Hot Module Replacement라고 부릅니다).

```
npm run dev
# ▲ Next.js
# - Local:    http://localhost:3000
# ✓ Ready in 3.9s
```

브라우저에서 `http://localhost:3000` 으로 들어가면 화면이 보입니다. 코딩하는 내내 커두는 게 보통입니다.

비유 — 개발 서버는 주방의 시식용 화구입니다. 빠르게 끓이고 맛 보며 간을 맞추는 단계. 손님에게 내는 화구는 아닙니다(=실서비스 아님).

2-2. 개발 서버에서 자주 만나는 동작

상황	의미
저장 즉시 브라우저가 갱신	HMR — 정상 동작
터미널에 빨간 에러	코드를 못 읽어서 그래요. 메시지 줄의 파일 경로로 가서 고치면 됩니다.
포트 3000 이미 사용중	다른 dev 서버가 떠 있는 것. 그 창을 닫거나 <code>npm run dev -- -p 3001</code> 로 다른 포트 사용.

3단계 · 빌드 (배포용으로 압축)

3-1. npm run build 의 의미

개발 모드 코드는 가독성이 좋게 “날것” 상태입니다. 빌드는 이것을 브라우저가 가장 빠르게 받아갈 수 있는 형태로 압축·최적화하는 과정입니다. Next.js는 이때 페이지 단위로 미리 만들 수 있는 것들을 미리 만들어 두기도 합니다.

```
npm run build
# ✓ Compiled successfully
# ✓ Generating static pages
```

비유 — 도시락 포장. 시식 화구에서 만든 음식을 통에 차곡차곡 담아 손님이 가져갈 수 있게 정리합니다.

실수 주의 사용자가 적어주신 `npm build` 는 잘못된 명령어입니다. 항상 `npm run build` 가 맞아요. `run` 이 빠지면 `npm`이 빌트인 명령어를 찾다 실패합니다(예외: `npm start` · `npm test` · `npm install` 은 `run` 을 생략 가능).

3-2. `npm start` 의 의미

빌드 결과물을 실제 서비스 모드로 띄우는 명령입니다. 개발 서버(dev)와 달리 코드 자동반영이 안 되는 대신 훨씬 빠르고 가볍습니다.

```
npm run build    # 먼저 빌드를 한 번 해두고
npm start       # 빌드 결과물로 서비스 실행
```

실제 배포 환경(예: Vercel)에서는 사람이 직접 이 명령을 치지 않습니다. 플랫폼이 알아서 `build` → `start`를 돌립니다. 로컬에서는 “배포 직전 마지막 점검”용으로 한 번씩 돌려보면 좋습니다.

4단계 · 디자인과 동작 (CSS · TSX)

1~3단계가 “도구 다루기”라면, 진짜 개발은 이 안에서 일어납니다. Next.js에서 화면 하나는 보통 한 개의 `.tsx` 파일로 만들어지고, 그 안에서 **CSS**로 꾸미는 일이 같이 진행됩니다.

TSX

화면의 구조 + 동작

TypeScript(타입 있는 JS) + JSX(HTML처럼 생긴 문법). “버튼이 있고 누르면 A가 일어난다”는 걸 한 파일에 적습니다.

CSS

화면의 모양 (색·여백·폰트)

전역 CSS 파일, CSS Modules, 또는 Tailwind 같은 유틸리티 클래스로 적용합니다. 이 프로젝트는 **Tailwind CSS**를 씁니다.

4-1. 아주 작은 예시

```
// app/page.tsx - “/” 경로의 첫 화면
export default function Home() {
  return (
    <main className="p-8">
      <h1 className="text-2xl font-bold text-blue-600">
        안녕하세요, Next.js!
      </h1>
      <p className="mt-2 text-gray-600">
        이 글자는 CSS(Tailwind)로 색·크기를 정합니다.
      </p>
    </main>
  );
}
```

- `<main>` · `<h1>` 같은 태그 = HTML과 동일한 의미. 구조입니다.
- `className="p-8 text-blue-600 ..."` = Tailwind CSS 클래스. 모양입니다.

- `export default function Home() { ... }` = 이 파일이 "Home"이라는 화면(컴포넌트)이라고 알리는 부분. 동작을 추가하려면 이 함수 안에 자바스크립트를 더 씁니다.

4-2. HTML 시절과 어떻게 다른가

	순수 HTML	Next.js
파일	<code>index.html</code>	<code>app/page.tsx</code>
실행	그냥 더블클릭	<code>npm run dev</code> 필요
스타일	<code>style.css</code> 직접 작성	Tailwind 클래스 / CSS Modules / 전역 CSS
동작	<code><script></code> 따로	같은 <code>.tsx</code> 안에 함수로 작성
여러 페이지	파일을 각각 만듦	폴더 구조가 곧 URL (<code>app/about/page.tsx</code> → <code>/about</code>)

5단계 · 한 장 요약 + 명령어 치트시트

언제	명령	한 줄 설명
처음 받아왔을 때	<code>npm install</code>	부품을 창고에 채우기
코딩하는 동안	<code>npm run dev</code>	개발 서버 — 저장하면 자동 새로고침
배포 전 점검	<code>npm run build</code>	최적화·압축
빌드 결과 실행	<code>npm start</code>	실제 서비스 모드로 띄우기
코드 점검	<code>npm run lint</code>	문법·스타일 자동 검사
타입 점검	<code>npm run typecheck</code>	TypeScript 오류 확인

이 프로젝트 특이사항 ICT 포털 프로젝트는 `npm` 대신 `pnpm` 을 사용합니다. 위의 모든 명령어를 `pnpm install`, `pnpm dev`, `pnpm build`, `pnpm start` 로 바꿔서 쓰시면 됩니다. (동작은 동일, 속도와 디스크 사용이 더 좋음)

그래서, 이게 개발의 “기본”인가?

네 — 정확히는 “프레임워크를 다루기 위한 기본 동작”입니다. 실제로는 이 안에서:

- 화면 만들기 — `app/` 폴더에 `.tsx` 파일을 추가
- 꾸미기 — Tailwind 클래스 또는 CSS 작성
- 데이터 가져오기 — API 호출, DB 연결
- 저장하기 — Git으로 버전 관리, 배포

이 네 가지가 매일 반복되는 진짜 작업이고, `install/dev/build/start`는 그 작업을 받쳐주는 기본 조작입니다. 자동차로 비유하면 시동·기어·주차 같은 운전의 기본 동작에 가깝습니다 — 운전(=코딩) 자체는 그 위에서 일어나죠.

보충편 · .env.local과 프로젝트 설치의 모든 것

왜 환경변수가 필요한지, 처음 받아온 프로젝트를 띄우기까지 무엇이 필요한지 — 초보자 시점에서 정리

1. 왜 .env.local 이 필요한가

웹 서비스 하나를 만들 때 코드 안에 자주 등장하는 두 종류의 “외부값”이 있습니다.

- **비밀(Secrets)** — API 키, DB 비밀번호, 토큰처럼 *외부에 노출되면 안 되는 값*
- **설정(Settings)** — DB 주소, 외부 서비스 URL처럼 *환경(개발/운영)마다 달라지는 값*

이 값들을 코드에 직접 적어두면 두 가지 문제가 생깁니다.

사고 1 코드를 GitHub에 올리는 순간 비밀이 전 세계에 공개됩니다. 키가 유출되면 누가 내 OpenAI 계정으로 수십만 원을 결제할 수도 있어요. 실제로 매일 수천 건의 키가 GitHub에서 자동 봇에게 수집됩니다.

사고 2 로컬 PC에선 개발용 DB, 운영 서버에선 진짜 DB를 써야 하는데, 주소를 코드에 박아두면 **매번 코드를 고쳐서 배포**하게 됩니다. 사람 손이 들어가면 사고가 납니다.

그래서 “코드 바깥”에 비밀과 설정을 따로 둡니다. 그 “바깥”이 바로 `.env.local` 같은 환경변수 파일입니다.

비유 — `.env.local` 은 내 PC에만 있는 비밀 노트입니다. 노트 자체는 GitHub에 절대 올리지 않고 (`.gitignore` 에 포함), 코드는 “노트의 OO 칸을 읽어라”라고만 쓰여 있어요. 다른 PC에서는 같은 노트의 다른 내용이 채워져 있어도 코드 수정 없이 동작합니다.

2. 파일 이름 규칙과 우선순위

Next.js는 다음 파일들을 자동으로 읽습니다. 같은 이름의 키가 여러 파일에 있으면 **위에 있는 파일**이 이깁니다.

파일명	용도	git에 올림?
<code>.env.local</code>	내 PC 전용. 실제 키·비밀번호가 여기 들어감.	❌ 절대 안 됨
<code>.env.development</code>	개발 모드 전체에 공통 적용되는 값	가능(비밀 없을 때)
<code>.env.production</code>	운영 모드 전체에 공통 적용되는 값	가능(비밀 없을 때)
<code>.env</code>	모든 환경의 기본값	가능(비밀 없을 때)
<code>.env.example</code>	“이 키들이 필요해요”라고 알려주는 샘플 (값은 빈 칸)	✅ 올림

실무 패턴은 단순합니다.

- `.env.example` 을 git에 올려서 팀원이 어떤 키가 필요한지 본다.
- 각자 자기 PC에서 `.env.example` 을 `.env.local` 로 복사한다.
- 자신의 진짜 키를 `.env.local` 에 채워 넣는다.
- `.env.local` 은 절대 git에 안 올라간다(`.gitignore` 덕분).

3. `NEXT_PUBLIC_` 접두사 — 클라이언트 노출의 경계선

웹 코드는 두 곳에서 돕니다.

- **서버(Server)** — Next.js 서버 안. 사용자는 코드를 볼 수 없음.
- **브라우저(Client)** — 사용자 브라우저. **JS 코드와 변수가 결국 사용자에게 다 보임.**

그래서 환경변수도 "브라우저에 보내도 되는 것"과 "서버에만 있어야 하는 것"을 구분합니다. Next.js는 이름 앞에 `NEXT_PUBLIC_` 이 붙어 있으면 브라우저로 보내고, 안 붙어 있으면 서버에만 돕니다.

```
# 브라우저에서도 읽을 수 있음 (사용자에게 노출됨)
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co

# 서버에서만 읽힘 (절대 브라우저로 안 감)
SUPABASE_SERVICE_ROLE_KEY=eyJ....비밀....
ANTHROPIC_API_KEY=sk-ant-...
```

철칙 비밀 키에는 **절대로** `NEXT_PUBLIC_` 을 붙이지 마세요. 한 번 빌드되면 사용자 브라우저로 그대로 흘러갑니다. 이 사고는 거의 모든 신입 개발자가 한 번씩 합니다.

비유 — 사무실 벽에 붙이는 안내문(`NEXT_PUBLIC_`)과 금고에만 두는 마스터키(접두사 없음). 안내문은 손님이 봐도 되지만, 마스터키는 절대 벽에 붙이면 안 됩니다.

4. 실제 `ict-portal`의 `.env.local` 읽어보기

지금 이 프로젝트의 `.env.local` 이 그룹별로 어떻게 묶여 있는지 살펴보면, 위 원리가 그대로 적용된 것을 알 수 있어요.

그룹	키 예시	의미
Supabase (DB)	NEXT_PUBLIC_SUPABASE_URL , NEXT_PUBLIC_SUPABASE_ANON_KEY , SUPABASE_SERVICE_ROLE_KEY	앞 두 개는 브라우저에서 "읽기 권한"으로 DB 접근. 마지막은 서버에서만 쓰는 "관리자 키".
OpenAI	OPENAI_API_KEY , EMBEDDING_MODEL	임베딩·문서 비전 파싱용. 서버에서만 사용.
Anthropic (Claude)	ANTHROPIC_API_KEY , LLM_MODEL	챗봇 LLM. 서버에서만 사용. claude-sonnet-4-6 호출.
Cohere	COHERE_API_KEY , RERANK_MODEL	RAG 단계의 리랭크용(현재 단순 챗팅 모드에서는 미사용).
법제처	LAW_GO_KR_API_KEY , LAW_GO_KR_BASE_URL	법령 검색 도구용 외부 API.
튜닝 파라미터	RETRIEVAL_TOP_K , RERANK_TOP_K	비밀이 아니라 "설정". 자주 바꾸며 실험하는 값.

같은 파일 안에 "브라우저로 보내도 되는 것"과 "서버에만 있어야 할 비밀"이 섞여 있고, 접두사 하나로 그 경계를 그은 것이 핵심입니다.

5. 새 PC에서 이 프로젝트를 처음 받아왔을 때 — 필수 단계

회사 PC가 바뀌었거나 팀에 새 멤버가 들어왔을 때, “코드만 받으면” 바로 안 됩니다. 다음 5단계를 거쳐야 화면이 뜹니다.

STEP 1

코드 받기

`git clone` 으로 저장소를 내려받는다. 또는 회사 NAS·USB로 받기.

STEP 2

Node.js + pnpm 설치

이 프로젝트는 Node 20 이상, pnpm 9가 필요. 설치 후 `node -v` · `pnpm -v` 로 확인.

STEP 3

의존성 설치

`pnpm install` . `package.json` 에 적힌 부품을 `node_modules` 에 받아 둬.

STEP 4

환경변수 채우기

`.env.example` 을 `.env.local` 로 복사하고, 각자 발급받은 API 키를 채워 넣는다.

STEP 5

DB 준비

Supabase 프로젝트가 이미 있으면 OK. 새로 만들었다면 `pnpm db:push` 로 스키마 적용.

STEP 6

개발 서버 띄우기

`pnpm dev` → `http://localhost:3000` 접속. 화면이 뜨면 성공.

5-1. 키는 어디서 받아오나요?

서비스	발급처	비고
Supabase	supabase.com → Project Settings → API	URL · anon key · service role key 세 가지
OpenAI	platform.openai.com/api-keys	크레딧 충전 필요. 키는 한 번만 표시되니 메모 필수.
Anthropic	console.anthropic.com → API Keys	<code>sk-ant-</code> 로 시작
Cohere	dashboard.cohere.com/api-keys	무료 tier에 trial key 있음
법제처	open.law.go.kr 가입 후 신청	승인까지 1~2일 걸릴 수 있음

6. 안전 수칙 — 사고를 미리 막는 법

- `.gitignore` 에 `.env*.local` 이 있는지 확인. 이게 없으면 git이 `.env.local` 을 같이 올려버립니다. 이 프로젝트는 이미 들어 있어요.
- 실수로 키를 커밋했다면 즉시 “회전(rotate)”. git에서 지워도 GitHub는 옛 커밋을 기억합니다. 키 발급처에서 새 키를 만들어 옛 키를 폐기하는 게 유일한 해결책.

- ❑ 팀원과 `.env.local` 을 공유할 때는 안전한 채널 사용. 카톡·메일은 부적합. 1Password·Bitwarden 같은 비밀 관리 도구나, Vercel 등 배포 플랫폼의 환경변수 기능을 쓰세요.
- ❑ 스크린샷·녹화 전 키 확인. 데모 영상에 키가 찍혀서 유출되는 사고가 흔합니다.
- ❑ 키 한 개로 모든 환경 쓰지 않기. 개발용·운영용 키를 분리하면 사고 영향 범위가 작아집니다.

7. 자주 만나는 에러와 해결

증상	원인	해결
<code>process.env.X is undefined</code>	키 추가 후 dev 서버를 재시작 안 함	Ctrl+C로 끄고 <code>pnpm dev</code> 다시 실행
브라우저에서 환경변수 못 읽음	<code>NEXT_PUBLIC_</code> 접두사 빠짐	이름 변경 후 재시작. 단, 진짜 비밀이면 절대 안 됨
값에 <code>\$</code> · <code>#</code> 이 들어 있는데 잘림	따옴표 없이 적었거나 주석으로 해석 됨	값을 큰따옴표로 감싸기: <code>KEY="값 #포함"</code>
API 호출에서 401 Unauthorized	키 오타·만료·권한 부족	발급처에서 새 키 생성 후 교체, 서버 재시작
Vercel 배포는 잘 되는데 로컬만 안 됨	Vercel 환경변수만 채우고 <code>.env.local</code> 은 안 채움	<code>vercel env pull</code> <code>.env.local</code> 로 동기화 가능

8. 한 장 요약 — 내가 PC 바뀌도 이대로 따라 하면 된다

```
# 1) 코드 받기
git clone <repo-url>
cd ict-portal

# 2) Node·pnpm 설치 (한 번만)
#   Node.js 20+ : https://nodejs.org
#   pnpm       : npm i -g pnpm@9

# 3) 의존성 설치
pnpm install

# 4) 환경변수 - .env.example를 복사하고 키 채우기
cp .env.example .env.local
#   → .env.local 을 열어서 ANTHROPIC_API_KEY, OPENAI_API_KEY 등 채움

# 5) (필요시) DB 스키마 적용
pnpm db:push

# 6) 개발 서버
```

```
pnpm dev
# → http://localhost:3000 에서 화면 확인
```

핵심 한 줄 `.env.local` 은 코드 바깥에 비밀과 설정을 두는 자리입니다. 그래서 코드는 같아도 PC마다 환경마다 다르게 동작할 수 있고, 키가 GitHub에 새지 않습니다. 새 PC에 프로젝트를 받았을 때 “화면이 안 떠요”의 90%는 이 파일을 안 만들었거나 키가 비어 있어서 그렇습니다.